第 21 章：API 与成本模型

版本: v2.0（润色版）
作者: 内容撰写专家（上下游篇）
状态: 润色完成
最后更新: 2026-03-23

【本章导读】

学习目标： - 掌握 LLM API 的三种调用模式及其适用场景 - 能够准确计算和预估 Token 成本 - 理解模型选型策略和缓存限流机制

核心内容： LLM API 调用是 Agent 系统的核心环节，直接影响用户体验和运营成本。本章从调用模式、成本计算、模型选型、缓存限流四个维度，系统讲解如何设计高效、经济、可靠的 API 调用策略。

21.1 API 调用模式

LLM API 的调用模式决定了请求与响应的交互方式，直接影响系统的响应性、并发能力和用户体验。

一、同步调用（Synchronous Call）

定义与特点:

同步调用是最基础的 API 交互模式：客户端发送请求后阻塞等待，直到收到完整响应才继续执行。

```
客户端 ──→ 发送请求 ──→ 阻塞等待 ──→ 接收响应 ──→ 继续执行
```

优点： - **编程简单**：代码逻辑线性清晰，易于理解和调试 - **状态管理容易**：无需维护请求状态，响应到达即可处理 - **错误处理直接**：异常在调用点直接捕获

缺点： - **阻塞等待**：等待期间线程/进程无法处理其他任务 - **资源浪费**：长任务会占用连接和资源 - **用户体验差**：前端界面可能长时间无响应

适用场景： - **简单任务**：响应时间在 5 秒以内的快速操作 - **单用户场景**：无并发压力的工具类应用 - **强依赖场景**：后续逻辑必须等待当前结果才能执行

漫剧案例应用：漫剧设定检索使用同步调用。当作者询问「主角轩辕墨的能力设定是什么」时，系统需要立即检索并返回结果，检索耗时仅 1-2 秒，同步调用简单可靠。

常见误区：❌ 将长任务（如章节生成，耗时 30-60 秒）也用同步调用，导致用户界面长时间卡顿。

二、异步调用（Asynchronous Call）

定义与特点：

异步调用将请求提交后立即返回，客户端通过回调函数、Promise 或轮询机制获取结果。

```
客户端 ──→ 提交请求 ──→ 立即返回任务 ID ──→ 继续执行其他任务 ↓ 后台处理 ↓
客户端 ←── 回调通知/轮询获取结果 ←── 处理完成
```

优点：- **不阻塞**：提交请求后可立即处理其他任务 - **并发能力强**：可同时发起多个请求 - **用户体验好**：界面保持响应，用户可做其他操作

缺点：- **编程复杂**：需要处理回调、Promise 或状态轮询 - **状态管理困难**：需要维护任务状态（pending/running/completed/failed） - **错误处理分散**：错误可能在回调中处理，调试困难

适用场景：- **长任务**：响应时间超过 10 秒的操作（如长文本生成） - **批量任务**：需要同时处理多个请求 - **多用户并发**：高并发场景下避免资源耗尽

漫剧案例应用：漫剧章节正文生成使用异步调用。作者提交「生成第 3 章」请求后，系统立即返回任务 ID 和预计完成时间，作者可以继续编辑其他章节。生成完成后，系统通过通知告知作者查看结果。

实践参数：- **轮询间隔**：3-5 秒（太短增加服务器压力，太长用户体验差） - **超时时间**：根据任务类型设置（章节生成 120 秒，大纲生成 60 秒） - **状态持久化**：任务状态存入数据库，支持服务重启后恢复

常见误区：❌ 认为「异步一定更快」——实际总耗时与同步相同，只是不阻塞客户端。

三、流式输出（Streaming Output）

定义与特点：

流式输出是 LLM 特有的调用模式：模型生成一个 Token 就返回一个，客户端可以实时看到生成进度。

`LLM 生成：漫 → 剧 → 是 → 一 → 种 → ... ↓ ↓ ↓ ↓ ↓ 客户端接收：漫 漫剧 漫剧是 漫剧是一 漫剧是一种 ...`

优点： - **用户体验极佳**：实时看到生成进度，无需等待完整响应 - **可提前中断**：发现输出方向不对可立即停止，节省后续 Token - **感知延迟低**：首字延迟（Time to First Token）通常<1 秒

缺点： - **编程复杂**：需要处理流式数据（SSE/WebSocket） - **无法一次性处理**：不能等完整响应后再解析，需要增量处理 - **网络敏感**：网络波动可能导致流中断

实现方式： - **SSE（Server-Sent Events）**：HTTP 长连接，单向推送，适合文本流 - **WebSocket**：双向通信，适合需要客户端反馈的场景 - **HTTP Chunked Transfer**：传统 HTTP 分块传输

节省成本机制： 流式输出本身不减少 Token 数量，但支持**提前中断**： - 用户发现前 100 Token 就偏离方向，可立即停止 - 避免生成 1000 Token 后才发现需要重写 - 实际可节省 30-50% 的无效生成成本

漫剧案例应用： 漫剧创意沟通使用流式输出。Agent 的思考过程和回复内容实时显示，作者可以看到 Agent 如何分析创意、追问关键设定。如果 Agent 理解偏差，作者可立即打断纠正。

实践参数： - **首字延迟**：目标<1 秒（通过优化网络和 Prompt 长度） - **Token 间隔**：通常 20-50ms/Token（取决于模型和负载） - **中断响应**：客户端发送中断信号后，服务端应在 1 秒内停止生成

常见误区： ❌ 认为「流式输出能减少总 Token」——实际 Token 数由生成内容决定，流式只是分多次返回。但通过提前中断可间接节省成本。

四、三种模式对比与选择

| 维度 | 同步调用 | 异步调用 | 流式输出 | |------|---------|---------|---------| | **响应时间** | <5 秒 | >10 秒 | 任意（首字<1 秒） | | **编程复杂度** | 低 | 中 | 中高 | | **用户体验** | 一般 | 好 | 极佳 | | **并发能力** | 低 | 高 | 中 | | **成本优化** | 无 | 无 | 支持提前中断 | | **典型场景** | 设定检索、简单问答 | 章节生成、批量处理 | 创意沟通、长文本生成 |

选择策略： 1. **任务耗时<5 秒** → 同步调用（简单可靠） 2. **任务耗时>10 秒** → 异步调用（避免阻塞） 3. **需要实时反馈** → 流式输出（用户体验优先） 4. **高并发场景** → 异步调用 + 任务队列

混合使用： 实际项目中常混合使用多种模式： - 漫剧设定检索：同步（快速） - 漫剧章节生成：异步（长任务） - 创意沟通对话：流式（实时反馈）

21.2 成本计算与优化

LLM API 按 Token 计费，成本是 Agent 项目的重要考量。

一、Token 计算基础

输入 Token 构成：`输入 Token = System Prompt + 用户输入 + 对话历史 + 工具描述 + Few-shot 示例`

输出 Token 构成：
`输出 Token = LLM 生成的全部内容（包括推理过程、最终答案）`

计费方式（以 GPT-4 为例）： - 输入 Token：$0.03 / 1K Token - 输出 Token：$0.06 / 1K Token（通常是输入的 2 倍）

精确计算工具： 使用官方 Tokenizer 库精确计算： - OpenAI：`tiktoken` 库 - Claude：`anthropic` 库内置计算 - 开源模型：使用模型对应的 Tokenizer

估算公式（快速估算）： - 英文：1 单词 ≈ 1.3 Token - 中文：1 汉字 ≈ 1.5 Token - 代码：1 行 ≈ 5-10 Token（取决于语言）

漫剧案例： 漫剧大纲生成的 Prompt 分析： - System Prompt：500 Token - 世界观设定：800 Token - 角色设定：600 Token - 创意描述：300 Token - Few-shot 示例：400 Token - **输入总计**：2600 Token - **输出预计**：1500 Token - **单次成本**：2.6 × 0.03 + 1.5 × 0.06 = $0.168

常见误区： ❌ 只估算输出 Token，忽略输入 Token——实际输入可能占 60% 以上。 ❌ 用字符数粗略估算——不同语言、不同内容 Token 密度差异大。

二、成本预估方法

单任务预估：`单任务成本 = 输入 Token × 输入单价 + 输出 Token × 输出单价`

多任务预估（完整流程）：`总成本 = Σ(各环节单任务成本 × 预期调用次数) × 缓冲系数`

隐藏成本： 1. **重试成本**：网络错误、限流导致的重试（×1.2-1.5 系数） 2. **工具调用**：检索、验证等辅助操作的 Token 消耗 3. **嵌入计算**：向量嵌入生成的成本（如使用托管嵌入 API） 4. **反思迭代**：Reflexion 模式的自我检查（可能增加 50-100% 成本）

缓冲系数： 建议预留 20-30% 缓冲，应对： - 用户修改需求导致的重新生成 - 模型输出质量不佳需要重试 - 意外情况（如限流切换模型）

漫剧完整流程成本预估：

| 环节 | 模型 | 输入 Token | 输出 Token | 单次成本 | 调用次数 | 小计 |
|------|------|-----------|-----------|---------|---------|------| | 创意沟通 | GPT-4 | 2000 | 1000 | $0.12 | 5 轮 | $0.60 | | 设定生成 | GPT-4 | 3000 | 2000 | $0.21 | 1 次 | $0.21 | | 大纲生成 | GPT-4 | 2600 | 1500 | $0.17 | 1 次 | $0.17 | | 细纲分解 | GPT-3.5 | 2000 | 1000 | $0.003 | 1 次 | $0.003 | | 章节生成 | GPT-3.5 | 1500 | 3000 | $0.006 | 10 章 | $0.06 | | **总计** | - | - | - | - | **$1.043** |

加上 30% 缓冲：**单部漫剧（10 章）约 $1.36**

常见误区：❌ 按理想情况预估（无重试、无修改）——实际成本往往是预估的 1.5-2 倍。❌ 忽略对话轮次——创意沟通可能 5-10 轮，每轮都消耗 Token。

三、优化策略

1. 缓存策略

缓存什么： - **常见问题标准回复**：如「如何修改设定」「支持哪些功能」 - **固定知识检索**：世界观规则、角色基础设定（不常变） - **重复计算结果**：相同输入的嵌入向量、相同 Prompt 的生成结果

缓存键设计： ```python

用输入 Prompt 的哈希值作为缓存键

cache_key = hashlib.md5(prompt.encode()).hexdigest() ```

缓存过期： - **TTL（Time To Live）**：设置 24 小时或 7 天，避免知识过期 - **主动失效**：相关知识更新时清除缓存（如角色设定修改）

缓存命中率： 目标 30-50%，过高（>70%）可能说明系统过于静态，过低（<20%）说明缓存策略不当。

质量指标： - 缓存准确率：>99%（缓存内容与最新数据一致性） - 缓存更新延迟：<5 分钟（数据更新到缓存失效的时间）

漫剧案例： 漫剧世界观规则（如「灵力等级分为 1-9 级」）存入缓存，相同查询不重复调用 LLM，节省 80% 的检索成本。

---

2. Prompt 压缩

压缩方法： - **移除冗余描述**：删除重复、啰嗦的指令 - **使用缩写**：首次定义后用缩写（如「世界观设定」→「WS」） - **精简 Few-shot**：从 5 个示例减到 2-3 个高质量示例 - **限制输出长度**：设置 `max_tokens` 避免生成长度失控

压缩前后对比： ``` 压缩前 System Prompt：800 Token 压缩后 System Prompt：500 Token 节省：37.5%

单次生成节省：800 × 0.03 / 1000 = $0.024 1000 次生成节省：$24 ```

质量权衡： 压缩后需 A/B 测试，确保质量指标下降<5%（如通过率从 95% 降至 90% 以内，或质量评分从 4.2 降至 4.0 以内）。如果质量下降超过 5%，需要重新设计 Prompt 而非简单删减。

---

3. 批处理

适用场景： - 多个相似任务可合并为一次调用 - 批量生成角色设定、批量检查章节错误

实现方式： ```python

串行调用（5 次）

for character in characters: result = llm.generate(prompt_template.format(char=character))

批处理（1 次）

batch_prompt = "请为以下 5 个角色生成设定：\n" + "\n".join(characters) batch_result = llm.generate(batch_prompt) ```

限制： - 受模型上下文窗口限制（通常 8K-128K Token） - 批处理过大可能导致质量下降（批量>10 个任务时，质量评分下降>0.5 分） - 错误处理复杂（一个失败可能影响整体）

质量指标： - 批量大小：5-10 个任务/批次（平衡效率与质量） - 质量损失容忍度：<10%（批量与单任务质量评分差异）

漫剧案例： 批量生成 10 个配角设定，合并为一次调用，Token 从 5000×10=50000 降至 30000，节省 40% 成本。

---

4. 模型分级

策略： - **关键任务用强模型**：创意、大纲用 GPT-4 - **次要任务用弱模型**：章节正文、格式检查用 GPT-3.5 - **降级方案**：强模型限流时自动切换弱模型

漫剧案例： - 创意沟通：GPT-4（需要理解复杂创意，意图理解准确率>90%） - 设定生成：GPT-4（确保一致性，幻觉率<2%） - 大纲生成：GPT-4（需要规划能力，结构合理

性评分>4.0/5.0） - 章节正文：GPT-3.5（量大，人工修改率<20%） - 格式检查：GPT-3.5（简单任务，格式错误检测率>95%）

成本对比： 全部用 GPT-4：$5.2/部 分级策略：$1.4/部 **节省 73%**

质量监控指标： - 弱模型输出抽检率：≥10%（随机抽检 GPT-3.5 输出） - 弱模型质量阈值：质量评分≥3.5/5.0（低于此值切换回 GPT-4） - 降级触发条件：API 成功率<80% 或响应时间>10 秒

优化效果总结:

| 优化策略 | 成本降低 | 质量影响 | 实施难度 | |---------|---------|---------|---------| | 缓存 | 20-40% | 无 | 低 | | Prompt 压缩 | 10-30% | 可能轻微 | 中 | | 批处理 | 20-50% | 可能轻微 | 中 | | 模型分级 | 50-70% | 可控 | 低 | | **综合优化** | **70-85%** | **可控** | **中** |

21.3 模型选型策略

选择合适的 LLM 模型，是在成本、质量、速度之间找到最佳平衡点。

一、模型能力对比

顶级模型（GPT-4、Claude-3 Opus、Gemini Ultra）： - **优势**：推理能力强、理解复杂指令、创意质量高 - **劣势**：成本高（$0.03-0.15/1K 输入 Token）、限流严格 - **适用**：创意生成、复杂规划、质量敏感任务

中端模型（GPT-3.5、Claude-3 Sonnet、Gemini Pro）： - **优势**：平衡型、成本低（$0.001-0.01/1K 输入 Token）、速度快 - **劣势**：复杂推理能力有限 - **适用**：一般对话、格式转换、知识问答

轻量模型（GPT-3.5 Turbo、各类 7B-13B 开源模型）： - **优势**：极低成本、快速响应、可本地部署 - **劣势**：能力有限、需要精细 Prompt - **适用**：简单任务、高频调用、预算有限场景

专项模型： - **代码专用**：CodeX、StarCoder（代码生成/解释） - **多模态**：GPT-4V、Claude-3 Vision（图像理解） - **长文本**：Claude-3（200K 上下文）、GPT-4 Turbo（128K）

二、成本与能力平衡

分级策略： `关键任务（创意/大纲）→ GPT-4 ↓ 限流/故障 降级任务（正文/检查）→ GPT-3.5 ↓ 仍失败 保底方案 → 返回缓存/提示用户稍后重试`

成本阈值： 设置单任务成本上限，超过则： - 用弱模型生成 - 或拒绝任务并提示用户

质量监控： - 定期抽样检查弱模型输出质量 - 用户反馈收集（满意度评分） - 自动评估（用强模型评估弱模型输出）

漫剧案例： 漫剧大纲生成默认用 GPT-4，当遇到限流时： 1. 第 1 次重试：等待 10 秒后用 GPT-4 重试 2. 第 2 次重试：等待 30 秒后用 GPT-4 重试 3. 第 3 次重试：切换到 GPT-3.5 并通知用户「为保证响应速度，本次使用标准模型」 4. 质量检查：用 GPT-4 快速评估 GPT-3.5 输出的大纲质量，如低于阈值则重新生成

三、场景化选型建议

| 场景 | 推荐模型 | 理由 | 备选方案 | |------|---------|------|---------| | **创意生成** | GPT-4/Claude-3 | 需要创造力和理解力 | GPT-3.5（质量略降） | | **事实问答** | GPT-3.5 | 知识型任务，轻量模型足够 | 轻量开源模型 | | **代码生成** | GPT-4/CodeX | 代码需要精确性 | Claude-3（代码能力强） | | **长文本处理** | Claude-3 | 200K 上下文优势 | GPT-4 Turbo + 分块 | | **多模态** | GPT-4V | 图像理解能力强 | Claude-3 Vision | | **高频调用** | GPT-3.5 Turbo | 成本低、速度快 | 本地部署开源模型 | | **企业私有** | 本地部署（Llama 2/3） | 数据不出域 | 托管 API + 数据脱敏 |

选型决策流程： `1. 任务类型分析（创意/知识/代码/多模态） ↓ 2. 质量要求评估（高/中/低） ↓ 3. 成本预算确认（单任务上限） ↓ 4. 延迟要求确认（实时/可接受延迟） ↓ 5. 选择模型 + 备选方案`

常见误区： ❌所有任务都用最强模型——成本过高，简单任务轻量模型就够了。 ❌为了省钱全用弱模型——关键任务质量下降导致返工，总体成本更高。 ❌只看价格不看场景——选错模型导致质量差，用户流失成本远高于 Token 成本。

21.4 缓存与限流

缓存和限流是保障系统稳定性和经济性的关键机制。

一、缓存策略深度设计

缓存层级： `L1 缓存（内存）：热点数据，TTL 5 分钟 ↓ 未命中 L2 缓存（Redis）：常用数据，TTL 24 小时 ↓ 未命中 L3 缓存（数据库）：历史数据，长期存储 ↓ 未命中 LLM API 调用`

缓存键设计进阶： ```python

简单哈希（完全匹配才命中）

cache_key = hash(prompt)

语义缓存（相似输入可命中）

将 Prompt 向量化，检索相似度>0.95 的缓存

prompt_embedding = embed(prompt) similar_cache = vector_db.search(prompt_embedding, threshold=0.95) ```

缓存失效策略： - **TTL 失效**：设置过期时间 - **主动失效**：相关知识更新时清除 - **LRU（Least Recently Used，最近最少使用）淘汰**：缓存满时淘汰最近最少使用

漫剧案例： 漫剧世界观规则缓存： - L1 缓存：当前会话中查询过的规则（内存，TTL 会话结束） - L2 缓存：所有已确认的规则（Redis，TTL 7 天，规则修改时主动失效） - 缓存命中率：85%，节省大量检索成本

二、API 速率限制

限流类型： - **RPM（Requests Per Minute，每分钟请求数）**： 每分钟请求数限制 - **TPM（Tokens Per Minute，每分钟 Token 数）**： 每分钟 Token 数限制 - **RPD（Requests Per Day，每日请求数）**： 每日请求数限制

识别限流： `python try: response = llm.generate(prompt) except RateLimitError as e: # HTTP 429 或错误信息包含 "rate limit" if e.status_code == 429 or "rate limit" in e.message: handle_rate_limit()`

限流监控： ```python

实时监控 RPM/TPM

current_rpm = count_requests_last_minute() current_tpm = count_tokens_last_minute()

if current_rpm > limit_rpm * 0.8: # 达到 80% 阈值 alert_admin() # 告警 pause_low_priority_tasks() # 暂停低优先级任务 ```

多 Key 轮换： ```python

多个 API Key 轮流使用

api_keys = [key1, key2, key3] current_key_index = 0

def get_next_key(): global current_key_index key = api_keys[current_key_index] current_key_index = (current_key_index + 1) % len(api_keys) return key ```

漫剧案例： 漫剧生成系统配置 3 个 API Key，每个 Key 限制 RPM=60。系统监控每个 Key 的使用情况，当某个 Key 达到 80% 时自动切换到下一个 Key，避免触发限流。

三、重试与降级

重试策略： ```python import time

def retry_with_backoff(func, max_retries=5): for attempt in range(max_retries): try: return func() except (RateLimitError, TimeoutError) as e: if attempt == max_retries - 1: raise # 最后一次重试失败，抛出异常

```
        # 指数退避：1s, 2s, 4s, 8s, 16s
        wait_time = (2 ** attempt) + random.uniform(0, 1)
        time.sleep(wait_time)
```

```

**最大重试次数**： - **限流错误**：3-5 次（给服务器恢复时间） - **超时错误**：2-3 次（可能是临时网络问题） - **其他错误**：0-1 次（可能是永久性问题）

**降级方案**： `python def generate_with_fallback(prompt, primary_model="gpt-4", fallback_model="gpt-3.5"): try: return llm.generate(prompt, model=primary_model) except RateLimitError: # 限流时降级 notify_user("系统繁忙，使用备用模型生成") return llm.generate(prompt, model=fallback_model) except Exception as e: # 其他错误返回缓存或提示 cached = get_cache(prompt) if cached: return cached raise`

**用户通知**： - **短暂延迟**（<10 秒）：不通知，用户无感知 - **中等延迟**（10-30 秒）：显示「系统繁忙，请稍等」 - **长时间延迟**（>30 秒）：提供选项（继续等待/使用备用模型/稍后重试）
```

漫剧案例： 漫剧章节生成遇到限流： 1. 第 1 次重试：等待 2 秒，用原模型重试 2. 第 2 次重试：等待 4 秒，用原模型重试 3. 第 3 次重试：等待 8 秒，用原模型重试 4. 降级：切换到 GPT-3.5，通知用户「为保证响应速度，本次使用标准模型」 5. 仍失败：返回错误，建议用户 5 分钟后重试

质量指标： - **服务可用性**：>99%（成功响应次数/总请求次数） - **重试成功率**：>80%（重试后成功的次数/总重试次数） - **降级触发率**：<5%（降级次数/总请求次数，过高说明容量不足） - **用户通知及时率**：100%（延迟>10 秒时必须通知）

常见误区： ❌ 无限重试——实际应该设置上限并降级，避免浪费时间和成本。 ❌ 被限流后立即重试——实际会加重限流，应该等待或切换 Key。 ❌ 降级不通知用户——实际应该告知，管理用户预期。

21.5 简单举例

案例设计

- **案例名称**：漫剧生成的成本估算与优化
- **涉及知识点**：API 调用模式、成本计算与优化、模型选型策略、缓存与限流
- **案例目标**：帮助理解如何系统性分析和优化漫剧生成的 API 成本
- **案例内容要点**：
- 场景描述：漫剧创作平台上线后，单部漫剧（10 章）生成成本高达$15，远超预算目标$5/部，需要系统性优化
- 技术应用：用 tiktoken 分析 Token 用量，正文生成从 GPT-4 降级到 GPT-3.5，启用缓存节省 40% 检索成本，Prompt 压缩节省 37.5% 输入成本，配置 3 个 API Key 轮换限流
- 效果说明：成本从$15/部降至$6/部（降低 60%），用户满意度从 4.5/5 降至 4.3/5 在可接受范围
- **注意事项**：不展开 tiktoken 的具体使用细节（见 21.2 节）

21.6 最佳实践与陷阱

最佳实践： 1. **成本监控常态化**：每日统计 Token 用量和成本，设置告警阈值 2. **分级策略标准化**：明确哪些任务用强模型，哪些用弱模型 3. **缓存优先**：能缓存的尽量缓存，缓存命中率目标 30-50% 4. **流式输出默认开启**：除非有特殊原因，否则默认用流式提升体验 5. **降级方案测试**：定期测试降级方案是否正常工作

常见陷阱：

1. **陷阱 1：忽略隐藏成本**
2. 问题：只计算直接调用成本，忽略重试、工具调用、嵌入计算

3. 避免：用缓冲系数（×1.3-1.5）或详细追踪所有调用
4. **陷阱 2：过度优化导致质量下降**
5. 问题：过度压缩 Prompt、过度降级模型，质量明显下降
6. 避免：A/B 测试，质量下降>10% 则回退优化
7. **陷阱 3：缓存过期导致错误**
8. 问题：缓存的知识过期，返回错误信息
9. 避免：设置合理 TTL，相关知识更新时主动失效
10. **陷阱 4：重试雪崩**
11. 问题：大量请求同时失败，同时重试导致服务器压力更大
12. 避免：指数退避 + 随机抖动（jitter）

知识来源: - OpenAI API 限流文档 - https://platform.openai.com/docs/guides/rate-limits - Anthropic API 文档 - https://docs.anthropic.com/claude/docs/rate-limits - LangChain Cost Tracking 官方文档 - https://python.langchain.com/docs/guides/cost_tracking

---

修改记录: - v2.0 (2026-03-23): 润色版 — 句子简化、删除重复、优化结构、统一语气 - v1.1 (2026-03-22): 根据编辑统筹意见修改 — 规范知识来源格式 - v1.0 (2026-03-22): 初稿完成